

AI Cycling Coach — System Design Document

Versión: 0.1 (borrador de diseño) **Fecha:** 2026-07-24 **Naturaleza:** Estudio de diseño + plan de ejecución. No es un plan fijo; es la fotografía de las decisiones de diseño actuales y de las que se tomarán en cada fase.

0. Resumen ejecutivo

El objetivo no es otro planificador de entrenamientos, sino el **entrenador de ciclismo con IA más avanzado posible**: se comporta como un coach humano de élite que conoce la literatura científica, aprende continuamente del usuario, explica todas sus decisiones, adapta el entrenamiento a diario y planifica a largo plazo, sin generar nunca entrenamientos aleatorios.

Principio arquitectónico central: **la IA conversacional NO decide el entrenamiento**. Las decisiones las toma un **motor de planificación** determinista y explicable, alimentado por un **motor fisiológico** (modelo grey-box) y un **gemelo digital** del ciclista. El LLM es la interfaz: explica, traduce intención y hace de tutor científico.

Decisiones de diseño cerradas

Dimensión	Decisión
Entregable	Documento de diseño + plan de ejecución (PDF)
Datos	APIs cuando sean baratas: Strava/Garmin/Wahoo, Intervals.icu/TrainingPeaks, wearables (Whoop/Oura/HRV)
Motor fisiológico	Grey-box : esqueleto mecánico validado + capa Bayesiana/ML que aprende desviaciones por usuario
Cold-start	Priors de literatura + Bayes jerárquico : import histórico + tests de campo
Planificador	Heurística + scoring + simulación (horizonte deslizante); RL como fase futura
RAG científico	Híbrido escalonado : reglas curadas ahora, RAG completo por fases
IA conversacional	Claude: explica + traduce intención a restricciones + tutor científico con citas
Arquitectura	Monolito modular con límites extraíbles; multiagente como evolución futura
Stack	Python + Postgres
Validación	Backtesting histórico + N-of-1 + métricas de producto + ablation
Orden de construcción	Fase 1 datos/integraciones → Fase 2 gemelo + motor fisiológico → Fase 3 planificador mínimo

1. Introducción

1.1 Visión

Un sistema que emula a un entrenador de élite. No un catálogo de entrenamientos, sino un **proceso continuo de decisión** que responde a la fisiología del ciclista en cada momento.

1.2 Filosofía: la planificación es un proceso, no un plan

Un plan no es una lista fija de sesiones. Es el estado de un sistema dinámico. Cada nueva información (una mala noche de sueño, un HRV bajo, un entrenamiento fallido) **cambia el estado** y provoca una **reoptimización del plan completo**, no solo de la siguiente sesión.

Ejemplo: el usuario duerme poco → baja la recuperación → sube la fatiga → el motor recalcula toda la semana, y puede modificar incluso el entrenamiento del sábado.

Analogía rectora: un **GPS que recalcula la ruta**. No replanifica solo el próximo giro; reevalúa el trayecto completo ante cada cambio.

1.3 Principios de diseño

1. **La fisiología manda.**
2. **La evidencia científica tiene prioridad.**
3. **El aprendizaje personalizado modula la evidencia general** (no la sustituye).
4. **Toda decisión debe explicarse.**
5. **El sistema nunca inventa fisiología.**
6. **El sistema declara su incertidumbre.**

Ejemplo de (6): "No estoy seguro de que tu FTP sea 320 W. Mi estimación es 317–323 W (90% de confianza)."

1.4 Diferenciación

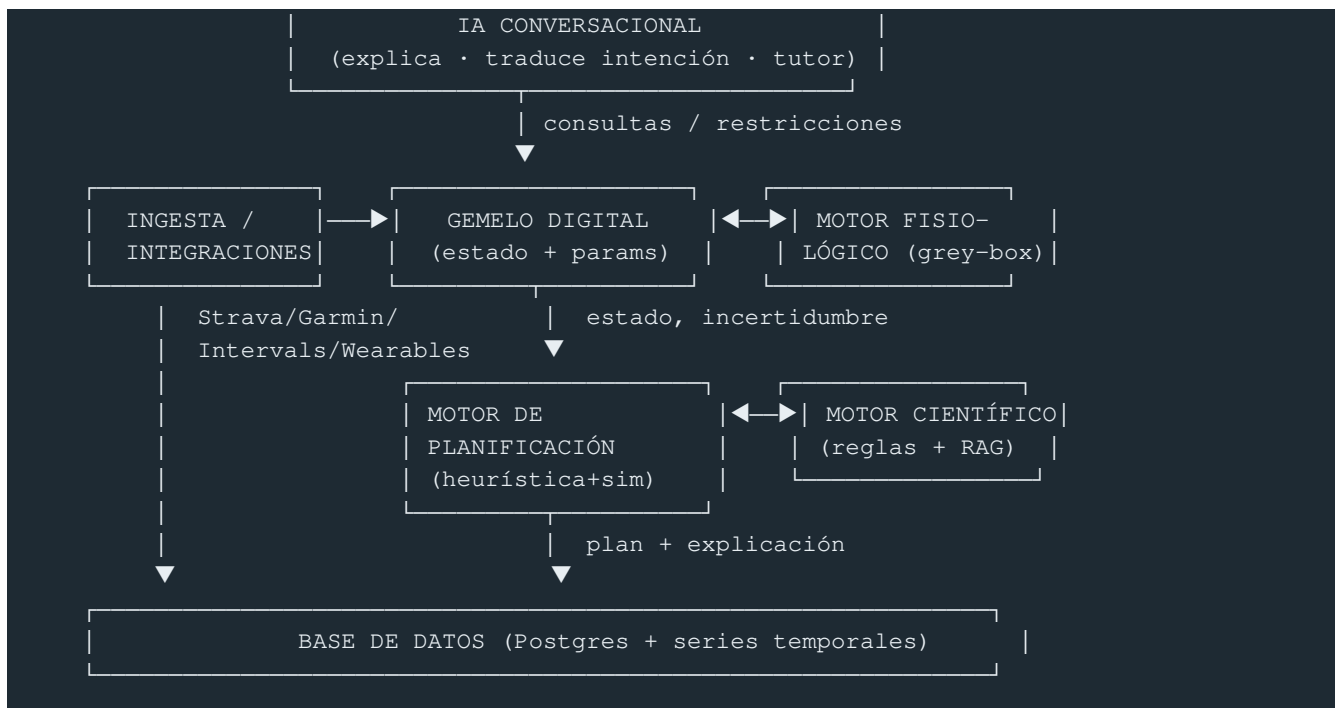
Frente a Join Cycling o TrainingPeaks, que adaptan la *sesión*, este sistema adapta la **estrategia completa**: mesociclo, periodización, tipo de estímulos, tapering, recuperación y carga — y además **aprende del usuario**.

1.5 Alcance de este documento

Cubre los 18 dominios del sistema con: propósito, decisiones de diseño, algoritmos/modelos, estructuras de datos clave y notas de fase. Prioriza concreción sobre extensión.

2. Arquitectura

2.1 Vista de componentes



2.2 Decisión: monolito modular con límites extraíbles

Un único servicio Python con módulos fuertemente separados por **interfaces explícitas** (contratos tipados). Motivos: un solo desarrollador, iteración rápida, evitar overhead de microservicios/agentes. Los límites se diseñan para poder extraer cualquier módulo a un servicio o agente independiente cuando lo pida la escala (esto habilita el cap. 14).

2.3 Contratos entre módulos (interfaces)

- `PhysiologyEngine.estimate_state(athlete_id, until) -> AthleteState` (con bandas de incertidumbre).
- `PhysiologyEngine.simulate(athlete_id, plan) -> PredictedTrajectory`.
- `Planner.replan(athlete_id, horizon, constraints) -> Plan + Rationale`.
- `Scientific.query(objective, context) -> WeightedEvidence`.
- `Conversational.interpret(user_msg) -> StructuredIntent | Explanation`.

Regla de oro: los módulos se comunican por **objetos de dominio tipados**, nunca por strings libres. El LLM vive en la frontera y **traduce** lenguaje natural a esos objetos.

2.4 Flujo principal (diario)

1. Llega dato nuevo (actividad, sueño, HRV, mensaje).
2. Se actualiza el **gemelo digital** (estado + re-estimación de parámetros).
3. El **planificador** dispara una reoptimización del horizonte.
4. Se genera el plan + la **explicación**.
5. Tras ejecutar, el sistema **compara predicho vs real** y **aprende**.

3. Gemelo digital

3.1 Concepto

El ciclista no se representa por 4 números (FTP/CTL/ATL/peso), sino por **cientos de variables** en cuatro capas, cada una con su dinámica temporal y su incertidumbre.

3.2 Taxonomía de variables

- **Permanentes** — edad, sexo, peso, altura, experiencia, historial deportivo, lesiones, material disponible.
- **Lentas** (semanas/meses) — FTP, eFTP, Critical Power (CP), W', VO₂max, economía, FatMax.
- **Estado diario** — CTL, ATL, TSB, sueño, HRV, FC reposo, fatiga, estrés, motivación, disponibilidad.
- **Ocultas (latentes, estimadas por IA)** — respuesta al sweet spot / VO₂ / polarizado, sensibilidad al calor, tiempos de recuperación VO₂ y FTP, capacidad de tapering, tolerancia al volumen.

Las variables ocultas **no existen al inicio**: se estiman y se afinan con el tiempo (ver cap. 9).

3.3 Estructura de datos

```
@dataclass
class Estimate:
    mean: float
    sd: float
    ci90: tuple[float, float]
    updated_at: datetime
    source: str
    # todo parámetro lleva incertidumbre
    # desviación (posterior bayesiana)

@dataclass
class AthleteState:
    static: dict
    slow: dict[str, Estimate]
    daily: dict[str, float]
    latent: dict[str, Estimate]
    as_of: datetime
    # permanentes
    # FTP, CP, W', VO2, ...
    # CTL, ATL, TSB, sueño, HRV, ...
    # variables ocultas
```

3.4 Decisiones de fase

- **Fase 1**: poblar `static` + `daily` desde ingesta; `slow` desde import histórico/tests.
- **Fase 2**: activar estimación bayesiana de `slow` y primeras `latent`.
- **Fase 3+**: refinar `latent` con el bucle de aprendizaje.

4. Motor fisiológico (grey-box)

4.1 Misión

No genera entrenamientos. Responde a una sola pregunta: *¿cómo está este ciclista ahora mismo, y cómo estará si hace X?* Entradas: potencia, FC, HRV, sueño, RPE, entrenamientos, competiciones, disponibilidad. Salidas: fitness, fatiga, recuperación, forma, FTP/VO₂ estimados, riesgo de lesión.

4.2 Arquitectura grey-box

Esqueleto mecanístico (white box) — modelos validados de la literatura.

- **Fitness–Fatiga (Banister/Busso):** respuesta al impulso con dos trazas. $\text{Perf}(t) = p_0 + k_1 \cdot \text{Fitness}(t) - k_2 \cdot \text{Fatigue}(t)$, con $\text{Fitness}(t) = \sum w(i) \cdot e^{\{-(t-i)/\tau_1\}}$, $\text{Fatigue}(t) = \sum w(i) \cdot e^{\{-(t-i)/\tau_2\}}$. Extensión Busso: ganancia de fatiga k_2 variable con la carga reciente.
- **Critical Power + W' balance (Skiba):** $P(t) = CP + W'/t$; depleción/recarga de W' intra-sesión con $\tau_{W'}$ dependiente de la intensidad.
- **DFA- $\alpha 1$** (de series RR) como marcador no invasivo de umbrales y de estado autonómico/fatiga.

Capa gris (aprendida) — un residuo que corrige lo que el esqueleto no explica, **por usuario:** $\hat{y} = f_{\text{mecanístico}}(x; \theta) + g_{\text{ML}}(x; \phi)$, donde g es pequeño y regularizado (GP o red ligera) y θ se estima por **Bayes jerárquico** (cap. 9). La capa gris nunca puede inventar fisiología imposible: se acota con restricciones físicas (monotonicidad, signos, rangos).

4.3 Estimación de estado

Filtro secuencial (Kalman/partícula o actualización bayesiana por lotes) que fusiona las señales ruidosas (potencia, HRV, sueño) en el `AthleteState`, propagando incertidumbre. Cada salida es un `Estimate`, no un escalar.

4.4 Riesgo de lesión

Modelo de riesgo basado en carga aguda/crónica (ACWR), monotonía/strain (Foster), rampas de volumen e histórico de lesiones. Salida = probabilidad calibrada + factores contribuyentes (para la explicación).

4.5 Decisiones de fase

- **Fase 2:** esqueleto mecanístico + estimación de estado con incertidumbre (sin capa gris).
- **Fase 3:** añadir capa gris g_{ML} y riesgo de lesión.
- **Futuro:** sustituir componentes por modelos superiores validados (ver cap. 18).

5. Estado de forma — CRI (Cyclist Readiness Index)

5.1 Motivación

No existe una única variable de "forma". Se propone un **índice compuesto propio**, el **CRI**, que combina varias dimensiones y cuyos **pesos son ajustables por usuario** (aprendidos con el tiempo).

5.2 Definición inicial (pesos por defecto)

```
CRI = 0.35·Rendimiento_reciente
      + 0.25·(1 - Fatiga_norm)
      + 0.15·Recuperación
      + 0.15·Tendencia
      + 0.10·Cumplimiento
```

Cada término normalizado a $[0,1]$. `Rendimiento_reciente` de records de potencia recientes vs esperados; `Fatiga/Recuperación` del motor fisiológico; `Tendencia` de la pendiente de fitness; `Cumplimiento` del

histórico de adherencia.

5.3 Personalización de pesos

Los pesos w se ajustan por usuario maximizando la correlación entre CRI y el rendimiento real observado (regresión regularizada con priors en los pesos por defecto → nunca se alejan sin evidencia).

5.4 Nota de diseño

El CRI es un **resumen para el usuario y una feature del planificador**, no una entrada al motor fisiológico (para no crear circularidad). Siempre se muestra con su incertidumbre.

6. Motor de planificación

6.1 El corazón del proyecto

No usa un LLM para inventar entrenamientos. Es un proceso determinista y explicable de **búsqueda + simulación + puntuación**.

6.2 Pipeline

```
Actualizar estado → Actualizar gemelo → Determinar objetivo fisiológico
→ Generar candidatos → Simular fisiología futura → Puntuar
→ Elegir mejor → Explicar → (ejecutar) → Aprender
```

6.3 Jerarquía de decisión (no elige "VO2" de golpe)

1. **Objetivo fisiológico** del bloque/día: VO₂, FTP, resistencia, sprint, capacidad anaeróbica, economía, recuperación.
2. **Tipo de estímulo** concreto que sirve a ese objetivo.
3. **Entrenamiento** concreto (composición de bloques, cap. 7).

6.4 Búsqueda con horizonte deslizante (analogía GPS)

En vez de optimizar solo mañana, genera **N trayectorias candidatas** de semanas (A, B, C...), **simula** para cada una con el motor fisiológico (FTP esperado, fatiga, prob. lesión, carga, cumplimiento esperado) y **elige la mejor** según la función de puntuación. Recalcula a diario (**receding horizon control** heurístico).

- Generación de candidatos: plantillas de periodización + perturbaciones guiadas por objetivo (no aleatorias) + restricciones duras (disponibilidad, eventos).
- Poda: se descartan candidatos que violan restricciones o superan umbrales de riesgo antes de simular.

6.5 Función de puntuación (multiobjetivo)

$$\text{Score} = \sum_k \beta_k \cdot \text{término}_k$$

Términos: alineación con objetivo, disponibilidad, fatiga, recuperación, proximidad de competiciones, historial/preferencias del usuario, evidencia científica, riesgo de lesión (penalización fuerte), monotonía (penalización), adaptación esperada. Los β_k son ajustables y en parte aprendidos. Se elige el mayor `score`; ante empates, el de **menor incertidumbre**.

6.6 Diseño preparado para RL (fase futura)

La misma abstracción (estado → acción → simulador → recompensa) permite sustituir la búsqueda heurística por una **política aprendida** más adelante. El simulador del motor fisiológico es, precisamente, el entorno que un agente RL necesitaría. Decisión: **no** empezar con RL; dejar la interfaz preparada.

6.7 Decisiones de fase

- **Fase 3:** planificador mínimo end-to-end (objetivo → 1 candidato razonable → explicación).
- **Fase 4:** búsqueda multi-candidato + simulación + puntuación completa.
- **Futuro:** RL / MPC formal.

7. Biblioteca de entrenamientos

7.1 Los entrenamientos no son etiquetas

"VO2" no significa nada por sí solo: existen cientos de sesiones distintas (30/15, 40/20, 30/30, 4×8, 5×5, 4×4, Billat, Rønnestad, microintervalos, over-under, piramidales...), cada una con adaptaciones diferentes.

7.2 Composición por bloques

No se guardan sesiones monolíticas, sino **bloques** reutilizables que se componen:

Warmup → Activación → Intervalos → Recuperación → Cooldown

Una sesión = secuencia parametrizada de bloques.

7.3 Metadatos por sesión/bloque

Tiempo >90% VO₂, tiempo en sweet spot, tiempo en Z5, duración, fatiga esperada, RPE esperado, carga cardiovascular / muscular / metabólica (tres ejes separados), terreno (rodillo/exterior), cadencia objetivo, nivel, tiempo mínimo, objetivo fisiológico primario/secundario.

7.4 Estructura de datos

```
@dataclass
class Block:
    kind: str          # warmup|activation|interval|recovery|cooldown
    duration_s: int
    target: dict        # %FTP, %CP, cadencia, ...
    repeats: int = 1

@dataclass
```

```
class WorkoutTemplate:
    id: str
    blocks: list[Block]
    objective: str
    meta: dict          # tiempos por zona, cargas 3-ejes, RPE esp., ...
    evidence_refs: list[str] # enlaces al motor científico
```

Los metadatos derivables (tiempo por zona, cargas) se **calculan** de los bloques, no se escriben a mano.

7.5 Decisiones de fase

- **Fase 3:** catálogo semilla de bloques + 15–20 plantillas cubriendo cada objetivo.
- **Fase 4:** generación paramétrica de variantes y cálculo automático de metadatos.

8. Modelos fisiológicos (detalle matemático)

Referencia de los modelos concretos usados por el cap. 4:

- **Impulso-respuesta (Banister):** dos exponenciales ($\tau_1 \approx 42$ d fitness, $\tau_2 \approx 7$ d fatiga como priors), ganancias k_1, k_2 por usuario.
- **Busso (fatiga variable):** k_2 función de la carga acumulada reciente $\rightarrow$ captura sobreentrenamiento.
- **Critical Power:** modelos de 2 parámetros (CP, W') y 3 parámetros; ajuste robusto sobre la curva de potencia-duración.
- **W' balance (Skiba):** recarga con $\tau_{W'} = f(D_{CP})$ (déficit respecto a CP).
- **DFA- $\alpha 1$:** cálculo desde intervalos RR con ventanas deslizantes; mapeo a umbrales aeróbico/anaeróbico.
- **Economía / FatMax:** de datos de potencia-FC y, si hay, metabólicos.
- **PerfPot (opcional):** alternativa a Banister para captar potencial de rendimiento.

Todos los parámetros se estiman con incertidumbre (cap. 9) y se acotan por rangos fisiológicos plausibles (principio 5: nunca inventar fisiología).

9. Aprendizaje personalizado

9.1 Idea

El sistema **compara lo esperado con lo obtenido** y corrige.

Esperaba RPE 6, obtiene 9 $\rightarrow$ el modelo estaba equivocado $\rightarrow$ corrige recuperación $\rightarrow$ corrige decisiones futuras.

9.2 Método: Bayes jerárquico (partial pooling)

- **Priors de literatura** para cada parámetro (τ , k , CP, W' , respuestas...).
- **Población $\rightarrow$ individuo:** cada usuario es un nivel; los parámetros individuales se encogen hacia la media poblacional (útil con pocos datos, evita sobreajuste). Con un solo usuario, domina el prior; con

mas voluntarios, emerge la estructura poblacional

- **Actualización secuencial:** cada sesión/día actualiza el posterior. La **incertidumbre** cae a medida que llegan datos → habilita el principio 6.

9.3 Variables ocultas

Las latentes (respuesta al SST/VO₂, tiempos de recuperación, tolerancia al volumen, sensibilidad al calor) se modelan como parámetros del modelo grey-box y se **infieren** del desajuste sistemático entre predicho y real.

9.4 Detección de modelo erróneo

Residuos estandarizados y calibración (¿el 90% de CI contiene el valor real el 90% de las veces?). Si la calibración se rompe, se re-estima o se amplía la capa gris.

9.5 Decisiones de fase

- **Fase 2:** actualización bayesiana de parámetros lentos.
- **Fase 3:** bucle esperado-vs-real sobre RPE/recuperación.
- **Fase 4+:** inferencia de latentes + jerarquía poblacional con voluntarios.

10. Motor científico (RAG + reglas)

10.1 Rol

Toda recomendación debe estar **respaldada por evidencia**, y la evidencia se **pondera** por nivel.

10.2 Jerarquía de evidencia

Metaanálisis ► Revisiones sistemáticas ► RCT ► Observacionales ► Casos

10.3 Diseño híbrido escalonado (decisión)

- **Ahora: base de reglas curada a mano** a partir de papers clave. Cada regla: condición → recomendación, con `peso_evidencia`, población, límites y referencias.
- **Arquitectura:** pensada desde el inicio para un **RAG completo** (ingesta de papers, extracción de población/resultados/limitaciones/nivel, generación semiautomática de reglas).
- **Crecimiento por fases:** el corpus curado se amplía; luego se automatiza la ingesta.

10.4 Estructura de una regla

```
@dataclass
class EvidenceRule:
    condition: str          # DSL o predicado sobre AthleteState/objetivo
    recommendation: str
    evidence_level: int     # 1=metaanálisis ... 5=caso
    population: str        # a quién aplica (para modular por usuario)
```

```
weight: float
refs: list[str]
```

10.5 Combinación con aprendizaje personalizado

La evidencia general (cap. 10) fija el **prior**; el aprendizaje personalizado (cap. 9) lo **modula** para el individuo — nunca lo contradice sin evidencia suficiente (principio 3).

11. IA conversacional

11.1 Rol (no decide)

Tres funciones, ninguna decisoria:

1. **Explica** las decisiones del motor ("¿por qué hoy sweet spot y no VO2?").
2. **Traduce intención** a restricciones estructuradas ("solo tengo 45 min" → `max_duration=45min`; "llueve" → `indoor=true`; "me voy de vacaciones" → bloqueo de fechas).
3. **Tutor científico**: responde dudas de fisiología apoyándose en el motor científico, **citando fuentes**.

11.2 Diseño

- LLM = **Claude** (Anthropic).
- **Patrón tool-use**: el LLM llama a herramientas tipadas (`get_plan_rationale`, `apply_constraint`, `query_evidence`) — no escribe entrenamientos.
- Toda cifra que muestre viene del motor, con su incertidumbre; el LLM no inventa números.
- Guardarraíl: si el usuario pide algo que contradice la seguridad fisiológica, el LLM explica el porqué del motor, no lo anula.

11.3 Decisiones de fase

- **Fase 3**: explicación de decisiones (`rationale` → lenguaje natural).
- **Fase 4**: traducción de intención (function calling) + tutor científico sobre el RAG.

12. Base de datos

12.1 Decisión: Postgres (+ series temporales)

- **Relacional (Postgres)**: usuarios, entrenamientos, plantillas, reglas, parámetros/posteriores del gemelo.
- **Series temporales**: streams de potencia/FC/RR/sueño/HRV (extensión tipo TimescaleDB o tablas particionadas).
- **Versiónado**: cada `Estimate` guarda `source` y `updated_at`; se conserva histórico de posteriores para auditar decisiones ("¿qué sabíamos el día que decidimos esto?").

12.2 Entidades núcleo

athlete, activity, stream, daily_metric, workout_template, block, plan, plan_decision (con rationale + evidencia usada), parameter_estimate, evidence_rule, feedback (RPE/sensaciones).

12.3 Trazabilidad

Cada `plan_decision` referencia el estado del gemelo, los candidatos evaluados, sus scores y la evidencia — condición necesaria para "explicar toda decisión" (principio 4) y para el ablation (cap. 17).

13. APIs e integraciones

13.1 Fuentes (priorizadas por coste/beneficio)

- **Strava** (OAuth): actividades + streams. Primera integración (import histórico del dueño).
- **Intervals.icu / TrainingPeaks**: métricas ya calculadas y estructura de sesiones.
- **Garmin / Wahoo**: datos del ciclocomputador.
- **Wearables (Whoop/Oura/HRV apps)**: sueño, HRV, FC reposo, readiness.

Criterio: integrar primero lo **barato y de alto valor**; el resto, por demanda.

13.2 Diseño

- Capa **adaptadora** por proveedor → normaliza a un **modelo de datos canónico** (el gemelo no conoce proveedores).
- Ingesta **incremental** (webhooks donde existan; polling donde no) + **backfill** histórico.
- Gestión de OAuth, rate limits y de-duplicación de actividades entre fuentes.

13.3 API interna

Contratos del cap. 2 expuestos como servicios internos; una API pública/UX se define en fase posterior.

13.4 Decisiones de fase

- **Fase 1**: adaptador Strava + modelo canónico + backfill. Luego Intervals/wearables.

14. Arquitectura multiagente (evolución futura)

Hoy: **monolito modular** (cap. 2). Cuando la escala lo justifique, los módulos con interfaces limpias se pueden promover a **agentes** cooperantes: agente de estado, de planificación, científico, conversacional. La decisión explícita es **no** pagar ese overhead al inicio, pero **diseñar los límites** para que la migración sea mecánica (mismos contratos, transporte por mensajes en vez de llamadas en proceso).

15. Machine Learning

15.1 Dónde vive el ML

- **Capa gris** del motor fisiológico (residuos acotados): GP o red ligera regularizada.
- **Inferencia bayesiana** de parámetros (PyMC/Stan): el "ML" central del proyecto al principio.
- **Personalización del CRI** y de los pesos β del scoring.
- **Riesgo de lesión**: clasificador calibrado.
- **Futuro**: política RL para el planificador.

15.2 Principios

- **Interpretabilidad y restricciones físicas** por encima de capacidad bruta.
- **Datos escasos** → priors fuertes, regularización, partial pooling.
- **Calibración** como métrica de primera clase (no solo error puntual).
- Todo modelo ML debe poder **explicar su contribución** (para el cap. 11 y 17).

16. Roadmap

Orden de construcción elegido: **Fase 1 (datos)** → **Fase 2 (gemelo+fisio)** → **Fase 3 (planificador)**. Nota de diseño: el **esquema del gemelo** se diseña en paralelo con la ingesta para no rehacerlo.

Fase	Entregable	Contenido
1. Datos	Ingesta + modelo canónico	Adaptador Strava, backfill, esquema Postgres, gemelo v0 (static+daily)
2. Fisiología	Estado con incertidumbre	Esqueleto mecanístico (Banister/Busso, CP/W', DFA- α 1), estimación bayesiana, CRI v1
3. Planificador mínimo	Sesión de mañana + explicación	Jerarquía objetivo→estímulo→sesión, 1 candidato, biblioteca semilla, explicación
4. Planificador completo	Simulación multi-candidato	Búsqueda con horizonte, scoring multiobjetivo, capa gris, riesgo lesión, intención NL, tutor RAG
5. Aprendizaje pleno	Personalización	Latentes, jerarquía poblacional (voluntarios), calibración continua
Futuro	Escala y frontera	RL/MPC, RAG automatizado, multiagente, modelos fisiológicos superiores

17. Validación

Con muestra pequeña al inicio, se combinan **cuatro estrategias** (todas elegidas):

1. **Backtesting histórico**: predecir sobre el historial de Strava (potencia/FTP futuro, RPE) y comparar con lo real. No requiere usuarios nuevos.
2. **N-of-1 / sujeto único**: diseño experimental con el dueño y voluntarios; medir mejora de FTP,

precisión de RPE y recuperación predichos vs reales, rendimiento en competición.

- 3. **Métricas de producto:** cumplimiento del plan, satisfacción, retención.
- 4. **Ablation de modelos:** comparar el grey-box completo contra versiones simplificadas (p. ej. solo Banister) para demostrar que cada componente aporta.

Métrica transversal: **calibración de la incertidumbre** (los CI cubren lo que dicen cubrir).

18. Futuras investigaciones

- **RL / MPC formal** para el planificador (el simulador ya es el entorno).
- **RAG científico automatizado** sobre miles de papers con extracción estructurada.
- **Arquitectura multiagente** real.
- **Modelos fisiológicos superiores:** sustituir componentes por lo mejor validado (glucógeno/ sustratos, termorregulación, modelos de sueño-recuperación).
- **Aprendizaje entre usuarios** (jerarquía poblacional rica) manteniendo privacidad.
- **Descubrimiento:** usar la capa gris para **generar hipótesis** fisiológicas nuevas y contrastarlas — la vía real para "superar el estado del arte".

Apéndice A — Trazabilidad principios → diseño

Principio	Dónde se garantiza
La fisiología manda	Motor fisiológico como fuente de verdad; LLM no decide (cap. 4, 11)
Evidencia con prioridad	Motor científico como prior ponderado (cap. 10)
Aprendizaje modula evidencia	Bayes jerárquico: prior=evidencia, posterior=individuo (cap. 9)
Explicar toda decisión	<code>plan_decision</code> con rationale + evidencia (cap. 6, 12)
Nunca inventar fisiología	Restricciones físicas en grey-box; rangos plausibles (cap. 4, 8)
Declarar incertidumbre	Todo parámetro es un <code>Estimate</code> con CI (cap. 3, 9)

Apéndice B — Infraestructura, costes y mantenimiento

Resumen: por la arquitectura elegida (grey-box + Bayes en vez de deep learning puro), **arrancar y validar cuesta casi nada**. El cómputo pesado es inferencia bayesiana, que es **CPU, no GPU**. El coste solo se vuelve relevante al comercializar, y ahí el reto es **legal (licencias de datos)**, no técnico.

B.1 Material / hardware

Recurso	¿Necesario?	Detalle
---------	-------------	---------

PC de desarrollo actual	Si, y basta	Ingesta y ajuste de modelos. Recomendable 16–32 GB RAM.
GPU	No	El grey-box + PyMC/Stan es CPU. Solo haría falta si se prueban redes neuronales grandes → se alquila por horas (spot), no se compra.
Servidor (VPS)	Solo al necesitar 24/7	2 vCPU / 4 GB sobra para el dueño + decenas de usuarios. Inferencia bayesiana por usuario = segundos-minutos, en batch nocturno.
Almacenamiento	Trivial	Años de streams de un ciclista ≈ MBs.

B.2 Coste por fase (€/mes, orientativo)

Fase	Usuarios	Coste técnico	Comentario
Desarrollo	1 + voluntarios	~0–15 €	Free tiers + open source + calderilla de LLM
Beta	10–50	~30–100 €	Postgres gestionado + hosting + LLM escalando
Comercial pequeño	500–2.000	~300–1.500 € + licencias de datos Δ	Coste técnico lineal; el riesgo es \$B.4

Partidas en fase desarrollo: Python/Postgres/PyMC/Stan/**pgvector** (0 € — el RAG vive **dentro** de Postgres, sin vector-DB aparte); Postgres gestionado free tier (Neon/Supabase); APIs Strava/Intervals/wearables gratis en uso personal.

B.3 LLM — único coste variable por usuario

El LLM **no decide** (solo interfaz), lo que ya evita el mayor gasto de un producto "todo-LLM". Palancas de control:

- **Modelo por tarea:** modelo barato/local para explicación e intención; modelo potente solo para el tutor científico. Ahorro 5–10×.
- **Prompt caching:** el contexto fijo (reglas, definición del sistema) no se paga repetido.
- Orden de magnitud con estas optimizaciones: **~0,10–1 € por usuario activo/mes.**

Estrategia de arranque: el LLM va detrás de una **interfaz agnóstica de proveedor** (coherente con "límites extraíbles", cap. 2). La decisión concreta del modelo se pospone; lo que importa es que sea intercambiable. Postura por defecto:

- **En desarrollo y beta** → **API** (Claude Haiku, o free tiers de Gemini/Groq). Con 1 usuario el coste es de céntimos/mes, la calidad es alta, el *function calling* es fiable y **no consume recursos del PC** (importante: el LLM local competiría por CPU/RAM con la inferencia bayesiana de PyMC/Stan, que corre en la misma máquina).
- **Modelo local (Ollama/llama.cpp)** → **opción, no default.** Se justifica solo con un motivo concreto: privacidad estricta, funcionamiento offline, o coste por usuario a **escala comercial** (donde el coste fijo de un servidor con GPU compensa el coste por-usuario de la API).

Referencia de un modelo local cuantizado Q4_7-8B ocupa ~4-5 GB en disco, necesita ~6-8 GB de RAM y va lento en CPU (~5–15 tok/s); mientras genera satura la CPU. No es "gratis" en recursos ni en tiempo de mantenimiento.

⚠ La **traducción de intención a restricciones** usa *function calling*; los modelos locales pequeños son menos fiables ahí. Es la primera tarea que conviene mantener en un modelo bueno (API).

B.4 Cuello de botella real: licencias de datos (estratégico)

- **Strava**: API gratis para uso personal, pero acuerdo comercial **restrictivo** (ha bloqueado apps de terceros). No asumir acceso comercial libre.
- **Garmin / TrainingPeaks**: programas de partner, a veces con coste/aprobación.
- **Intervals.icu**: mucho más abierto → aliado para empezar.

Para desarrollo y NoF-1 con voluntarios que dan permiso es **gratis**. La vía comercial de datos es una decisión de negocio separada, a validar con cada proveedor.

B.5 Mantenimiento

Técnico (recurrente):

- **Integraciones que se rompen** (Strava/Garmin cambian API/OAuth) → revisión de adaptadores (cap. 13). Es lo más frecuente.
- Refresco de tokens OAuth, reintentos, de-duplicación de actividades.
- Backups de BD, actualización de dependencias, monitorización.
- Jobs de re-ajuste bayesiano nocturno y **recalibración** de modelos (cap. 17).

Científico (lo diferencial):

- Mantener actualizada la base de reglas/evidencia (cap. 10).
- Vigilar calibración y deriva de los modelos.

Coste del mantenimiento: en dinero, básicamente el hosting fijo (~30–100 €/mes en beta). El coste dominante para un dev en solitario es **el tiempo propio**, no el dinero.

B.6 Conclusión

Arrancar y validar: < **15 €/mes** (PC + free tiers + API de LLM barata, con coste de céntimos a 1 usuario). El coste solo importa al comercializar, y el obstáculo es **licencias de datos**, no hardware ni cómputo — que el diseño grey-box mantiene deliberadamente baratos.